Dijkstra's Algorithm and Priority Queues

Contents

1. Overview 2
2. When moves have different lengths 3
   - 2.1 Diagonal moves: the 8-connected grid 3
   - 2.2 Edge costs: the fewest moves is not the shortest 5
3. Dijkstra's algorithm 6
   - 3.1 The fire picture 6
   - 3.2 The algorithm: settle the nearest, update its neighbours 7
   - 3.3 Why the nearest unsettled vertex is final 9
   - 3.4 Stopping at the goal: uniform-cost search 10
4. Dijkstra's algorithm on the room grid 10
5. The priority queue: finding the smallest fast 12
   - 5.1 Why a plain list is slow 12
   - 5.2 The binary heap 13
   - 5.3 Dijkstra's algorithm with a heap 15
6. Summary 17
7. Sources 17
8. Key terms 19

Where this fits:

- **Builds on:** Big-O notation; Breadth-first search.
- **Leads to:** A* search; Backward and bidirectional search; Fast marching and the Eikonal equation; Value iteration on a grid; Travel-time edge costs.
- **Compare with:** A* search; Value iteration on a grid.

1. Overview

Key point: Once diagonal moves are allowed, routes with the same number of moves differ in length, so the fewest moves is no longer the shortest route. Dijkstra's algorithm always settles the vertex nearest the start next, using a priority queue kept as a heap, and finds the shortest route to the door: 3.414 m, where breadth-first search returns 3.828 m.

So far the robot can turn its room into a graph and search it: on the grid of 0.5 m moves left, down, up and right, breadth-first search (G-2550) finds the route to the door with the fewest moves, 8 moves, 4.0 m. What still breaks: our robot can spin on the spot and then drive in any direction, so it can also cut across a cell diagonally, 0.707 m instead of two moves of 0.5 m.

Figure 1 shows what breadth-first search does once diagonal moves are allowed. Breadth-first search returns a route of 6 moves, 3.828 m (orange, dashed), but another route of 6 moves is only 3.414 m long (green). Breadth-first search counts moves, so to it the two routes are equal, and it keeps whichever it finds first.

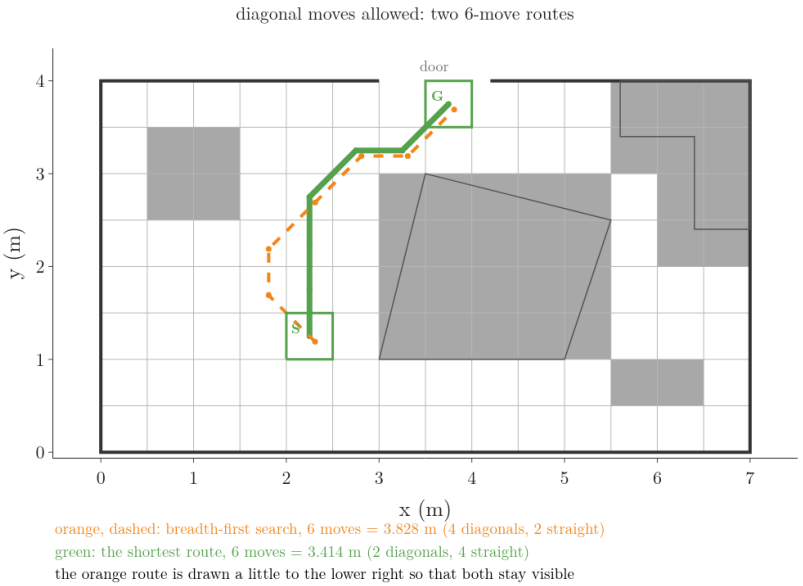


Figure 1: The room's 0.5 m grid with diagonal moves allowed. Orange, dashed: the route breadth-first search returns, 6 moves = 3.828 m, with 4 diagonals and 2 straight moves; green: the shortest route, also 6 moves, 3.414 m, with 2 diagonals and 4 straight moves. The two routes share their last 3 moves; the orange one is drawn slightly to the lower right so both stay visible

We find the shortest route by counting metres instead of moves; every number below is computed in the notebook `RO-033-dijkstra-and-priority-queues.ipynb`:

- the grid with diagonal moves, and graphs whose edges have costs (Section 2);
- Dijkstra's algorithm, first as a fire spreading through the waypoint graph, then as steps, why it is right, and how it stops at the goal (Section 3);
- Dijkstra's algorithm on the room grid (Section 4);
- the priority queue that makes it fast, kept as a binary heap (Section 5).

2. When moves have different lengths

> **Key point:** Allowing diagonal moves of 0.707 m beside straight moves of 0.5 m gives each edge its own cost. The length of a route is then the sum of its edge costs, and two routes of 6 moves can differ by 0.414 m.

2.1 Diagonal moves: the 8-connected grid

To reach the door by a shorter route, the robot must be allowed to cut corners of the grid. On the 4-connected grid (G-2544), where each cell is joined only to the cells sharing a side with it, going one cell right and one cell up takes two moves, 1.0 m. Our robot can turn on the spot to face the diagonal and drive straight there instead.

Figure 2 shows the moves this adds. From each cell the robot may now move to any of the 8 cells around it: 4 straight moves of 0.5 m and 4 diagonal moves. A diagonal move may not cut the corner of a blocked cell, because the robot would clip that cell on its way.

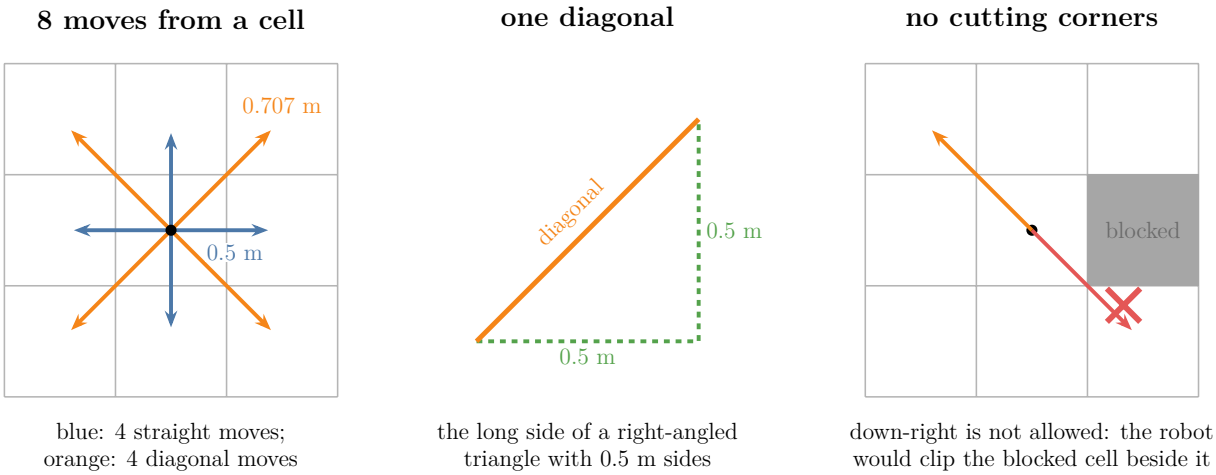


Figure 2: Left: from a cell, 4 straight moves of 0.5 m (blue) and 4 diagonal moves of 0.707 m (orange). Middle: one diagonal is the long side of a right-angled triangle whose two short sides are 0.5 m. Right: a diagonal move down and to the right next to a blocked cell is crossed out, because the robot would clip the blocked cell

Joining each cell to all 8 cells around it gives an **8-connected grid** (G-2564) (CMU 16-350, heuristics). With the rule against cutting corners, our room's 8-connected grid has the same 76 vertices as before and 190 edges instead of 114 (notebook).

How long is a diagonal move? It is the long side of the right-angled triangle in the middle of Figure 2, so Pythagoras' rule gives it; its length is the Euclidean distance (G-715), the straight-line distance, between the two cell centres. The symbols:

- a is the side of a cell, 0.5 m, which is also the length of a straight move;
- ℓ is the length of a diagonal move, in m.

Our numbers first, then the same steps in symbols:

$$\ell^2 = 0.5^2 + 0.5^2$$

$$\ell^2 = a^2 + a^2$$

$$\ell^2 = 0.25 + 0.25 = 0.5$$

$$\ell^2 = 2a^2$$

$$\ell = \sqrt{0.5} = 0.707 \text{ m}$$

$$\ell = \sqrt{2}\, a$$

A diagonal move covers the same ground as one move right and one move up, 1.0 m, in 0.707 m. The grid now has two kinds of moves, and that breaks breadth-first search.

2.2 Edge costs: the fewest moves is not the shortest

Breadth-first search treats every move as one step, but the robot cares about metres. In Figure 1 its route and the shortest route both have 6 moves. Adding up the metres, with 0.707 m for each diagonal move and 0.5 m for each straight move, the route breadth-first search returns, 4 diagonal and 2 straight moves:

$$4 \times 0.707 + 2 \times 0.5$$

$$= 2.828 + 1.0 = 3.828 \text{ m}$$

The shortest route, 2 diagonal and 4 straight moves:

$$2 \times 0.707 + 4 \times 0.5$$

$$= 1.414 + 2.0 = 3.414 \text{ m}$$

A graph in which every edge carries a number, its cost, is a **weighted graph** (G-2563), and the cost of a path (G-2543), a chain of edges, is the sum of the costs of its edges (LaValle §2.2.2). Here an edge's cost is its length in metres: 0.5 for a straight move, 0.707 for a diagonal. A cost could also be time or energy.

Breadth-first search goes wrong because it counts edges. Breadth-first search finishes all routes of 5 moves before any of 6, but among routes of 6 moves it cannot tell 3.414 m from 3.828 m; it keeps the first it finds, and which one that is depends on the order of the moves. The waypoint graph, whose edges have lengths, shows the same thing with two routes of 2 edges each:

- S–D–G, by the table's top-left corner:

$$2.500 + 1.005 = 3.505 \text{ m}$$

- S–P–G, by the pink pole:

$$2.236 + 2.786 = 5.022 \text{ m}$$

Breadth-first search returned S–D–G only because D comes before P in alphabetical order; trying the neighbours in reverse order, it returns S–P–G, 5.022 m (notebook). We need a search that counts metres, not moves.

3. Dijkstra's algorithm

> **Key point:** Dijkstra's algorithm settles the vertices in order of their distance from the start: it always takes the unsettled vertex with the smallest distance found so far, which is then final, and offers each neighbour the route through it. On the waypoint graph it settles the door at 3.505 m by S–D–G.

3.1 The fire picture

To count metres, picture the waypoints as oil depots and the edges as pipelines (NPTEL DAA, Dijkstra). We set fire to the depot at S at time 0. The fire runs along every pipeline at the same speed, 1 m per second, and each depot catches fire when the first flame reaches it.

All flames move at the same speed, so the first flame to reach a depot came by the shortest route. A depot's time of catching fire, in seconds, is therefore its shortest distance from S, in metres (DPV §4.4.1).

Figure 3 plays the fire on the waypoint graph. Burnt parts of pipelines are red, and each burnt waypoint shows the time it caught fire.

Under the room, a list keeps, for every waypoint not yet on fire, the arrival time of the first flame now on its way, smallest first. Such a list, which always hands out its smallest entry first instead of its oldest, is a priority queue (G-2568). The queue's first entry is always the next waypoint to catch fire. Watch two frames in particular:

- frame "t = 2.236 s: P catches fire": a flame starts from P towards the door G, due at 5.022 s, so G's entry is 5.022;
- frame "t = 2.500 s: D catches fire": a flame from D is due at G after 1.005 m more, at 3.505 s, so G's entry drops from 5.022 to 3.505.

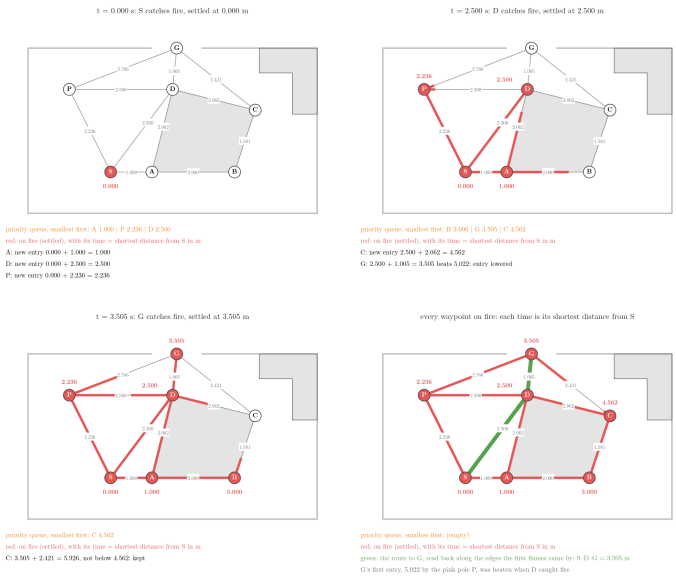


Figure 3: The waypoint graph as a fire spreading from S at 1 m per second along every edge. Burnt parts of edges are red; each burnt waypoint is filled red and shows the time it caught fire. Under the room: the priority queue, the arrival time of the first flame on its way to each waypoint not yet on fire, smallest first, and what the latest waypoint to catch fire changed. S catches fire at 0, A at 1.000 s, P at 2.236 s, D at 2.500 s (G's entry drops from 5.022 to 3.505), B at 3.000 s, G at 3.505 s and C at 4.562 s. The last frame shows every waypoint on fire, with the route S–D–G = 3.505 m to the door in green

The times at which the waypoints catch fire are their shortest distances from S:

Waypoint	S	A	P	D	B	G	C
Distance (m)	0	1.000	2.236	2.500	3.000	3.505	4.562

The route to the door is read back along the pipelines the first flames came by: G caught fire from D, and D from S, so S–D–G, 3.505 m.

3.2 The algorithm: settle the nearest, update its neighbours

A program cannot let a fire burn, but it does not need to: only the moments when a waypoint catches fire matter, and the next such moment always belongs to the waypoint with the earliest flame on its way, the first entry of the queue. Jumping from one such moment to the next is the algorithm. The symbols:

- u is the vertex just settled, such as D;
- v is one of its neighbours, such as G;
- $g(v)$ is the distance of v from S found so far, in m, such as 5.022 for G before D is settled;

- $c(u, v)$ is the cost of the edge from u to v, such as 1.005 m from D to G.

The steps (LaValle §2.2.2; DPV §4.4.1):

1. Set the start's distance $g(S)$ to 0 and put S in the queue; every other vertex has no distance yet.
2. Take the vertex u with the smallest $g(u)$ off the queue. The vertex u is now settled: $g(u)$ is its shortest distance.
3. For each neighbour v of u that is not settled: if the route through u is shorter than $g(v)$, set $g(v)$ to it, record u as the parent (G-2576) of v, the vertex it was reached from, and put v in the queue with its new distance.
4. Go back to step 2 until the queue is empty.

These four steps are **Dijkstra's algorithm** (G-2565), published by Edsger Dijkstra in 1959 (Wikipedia, Dijkstra's algorithm). Dijkstra's algorithm is the search loop with a frontier and a visited list with one change: the frontier (G-2547), the vertices waiting to be expanded, gives out the vertex with the smallest distance, not the oldest or the newest (LaValle §2.2.2). The settled vertices are its visited list (G-2548), the vertices already expanded.

Step 3 offers each neighbour the route through the vertex just settled and keeps the shorter of the two. When D is settled at 2.500, step 3 offers G the route through D. Our numbers first, then the same step in symbols:

$$g(G) = \min(5.022,\ 2.500 + 1.005)$$

$$g(v) = \min\big(g(v),\ g(u) + c(u, v)\big)$$

$$g(G) = \min(5.022,\ 3.505)$$

$$g(G) = 3.505$$

Here min means the smaller of the two numbers. The whole run, one row per settled vertex (notebook):

Settled	at (m)	Queue after step 3
S	0	A 1.000, P 2.236, D 2.500
A	1.000	P 2.236, D 2.500, B 3.000
P	2.236	D 2.500, B 3.000, G 5.022
D	2.500	B 3.000, G 3.505 (lowered from 5.022), C 4.562
B	3.000	G 3.505, C 4.562
G	3.505	C 4.562
C	4.562	(empty)

Some of step 3's offers lose, such as these two. Settling A offers D the route S–A–D:

$$1.000 + 2.062 = 3.062 \text{ m}$$

The route through A is longer than D's 2.500, so D keeps its entry. Settling B offers C the route through B:

$$3.000 + 1.581 = 4.581 \text{ m}$$

The route through B is longer than C's 4.562, so C keeps its entry. Step 3 lowers an entry only once: G's, when D is settled.

3.3 Why the nearest unsettled vertex is final

Step 2 settles D at 2.500 and never looks at it again. Could a route found later still reach D more cheaply? No; here is why.

When D is settled, the settled waypoints are S, A and P, and the queue holds D 2.500, B 3.000 and G 5.022. Any other route from S to D has to leave the settled waypoints somewhere, through a waypoint still in the queue, such as B. The symbols:

- u is the first waypoint not yet settled on that other route, such as B;
- $g(u)$ is its entry in the queue, 3.000 for B;
- r is the length of the rest of the route, from u on to D, in m: at least 0, because no edge is shorter than 0.

Our numbers first, then the same steps in symbols:

$$\text{other route} = 3.000 + r$$

$$\text{other route} = g(u) + r$$

$$3.000 + r \geq 3.000$$

$$g(u) + r \geq g(u)$$

$$3.000 \geq 2.500$$

$$g(u) \geq g(D)$$

So no later route can beat 2.500: D's entry was the smallest in the queue, and every other route already costs at least the entry of the waypoint where it leaves the settled ones (DPV §4.4.2; Wikipedia, Dijkstra's algorithm). Taking the best-looking step now and never revisiting it goes wrong in many problems; for Dijkstra's algorithm the argument above proves it right.

The argument needs every edge cost to be at least 0. Lengths always are, but a cost can be negative in other problems, such as a taxi trip that earns money (DPV §4.6.1). Then a later route can undercut a settled vertex, and Dijkstra's algorithm can return a wrong answer; the Bellman–Ford algorithm handles negative costs (Wikipedia, Dijkstra's algorithm).

3.4 Stopping at the goal: uniform-cost search

The robot needs the distance to the door, not to every waypoint. Since a settled distance is final (Section 3.3), the search can stop as soon as the door is settled: on the waypoint graph that is the 6th settled waypoint, and C is never settled.

The test must come when the door is taken off the queue, not when it first enters it. G first entered the queue at 5.022, when P was settled, and that entry was wrong; only when G comes off is 3.505 sure (CS188 §1.3.3; Red Blob, introduction). Dijkstra's algorithm stopped this way when the goal is settled is **uniform-cost search** (G-2567), the name artificial intelligence books use (CS188 §1.3.3; Wikipedia, Dijkstra's algorithm).

4. Dijkstra's algorithm on the room grid

> **Key point:** On the grid with diagonal moves, Dijkstra's algorithm settles the door's cell as the 54th cell, at 3.414 m, by a route of 2 diagonal and 4 straight moves: 0.414 m shorter than breadth-first search's route of the same 6 moves.

Back to the opening problem: the grid of Figure 1, with diagonal moves. Figure 4 runs uniform-cost search on it, one frame each time the settled distances pass another 0.5 m. Blue cells are settled and show their shortest distance from S; orange cells wait in the queue with their distance so far (bold).

Watch the blue region grow in rings of equal distance: in frame "settled up to 1.0 m", the cells straight above and below S are settled at 0.50 and 1.00, and the cells diagonally next to S at 0.71.

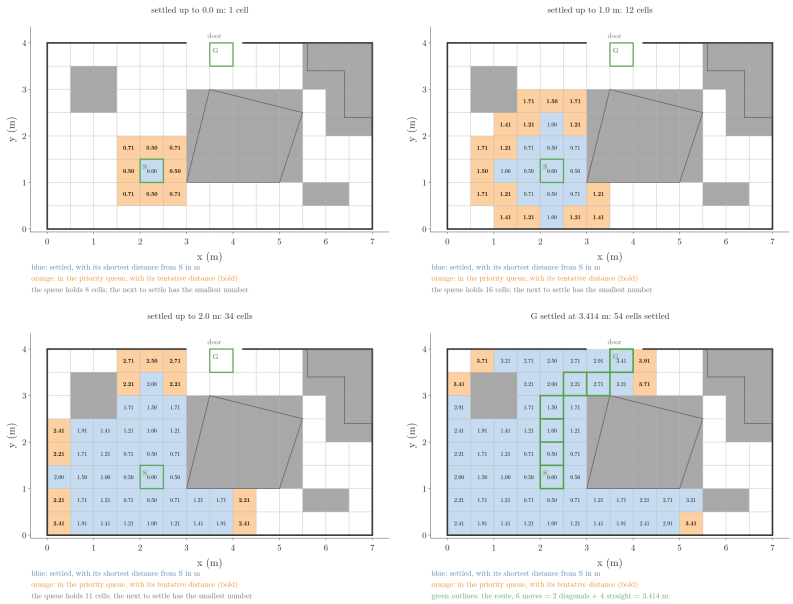


Figure 4: Dijkstra's algorithm on the room grid with diagonal moves, one frame each time the settled distances pass another 0.5 m. Blue: settled cells with their shortest distance from S in m; orange: cells in the priority queue with their distance so far, in bold. After 1.0 m, 12 cells are settled and 16 wait in the queue; after 2.0 m, 34 are settled. The last frame: the door's cell G is settled at 3.414 m as the 54th cell, and the route, outlined in green, runs 3 cells straight up from S, then diagonally, right and diagonally to G

The door's cell comes off the queue as the 54th settled cell, at 3.414 m; two other cells are also 3.414 m from S, and equal distances come off in the order they entered the queue (notebook). The route read back along the parents goes 3 cells up from S, one diagonal, one move right and one diagonal: 2 diagonal and 4 straight moves (notebook). The opening problem has its answer, a route shorter than breadth-first search's by:

$$3.828 - 3.414 = 0.414 \text{ m}$$

The rings are rounder than breadth-first search's rings of equal moves, because a diagonal now counts 0.707 m, not one full move. Without the stop at the door the search would settle all 76 free cells, the farthest 8.121 m from S (notebook).

The queue never held more than 16 cells here, so picking its smallest entry was quick. On a large map the queue holds thousands of entries, and picking the smallest one each time is where the work goes.

5. The priority queue: finding the smallest fast

> **Key point:** Dijkstra's algorithm needs a queue that hands out its smallest entry. A plain list makes either inserting or removing slow, about N^2 steps for N entries; a binary heap does both in about $\log_2 N$ steps, so a million entries take about 0.2 s instead of hours.

5.1 Why a plain list is slow

Step 2 of Section 3.2 removes the entry with the smallest distance, again and again, while step 3 keeps inserting new ones. A collection that supports exactly these two operations, insert an entry with a priority and remove the entry with the best priority, is a **priority queue** (G-2568) (DPV §4.5; Wikipedia, Priority queue). For Dijkstra's algorithm the best priority is the smallest distance.

The two obvious ways to keep one in a list each make one operation slow (Wikipedia, Priority queue):

Kept as	Insert	Remove the smallest
unsorted list	1 step: add at the end	look at all N entries
sorted list	walk up to N entries to its place	1 step: take the first

For N inserts and N removals, either list costs about N times N steps. A grid of 10^6 cells, such as a 500 m by 500 m floor in 0.5 m cells, then needs about 10^{12} steps; at about 10^8 simple steps per second (USACO Guide, time complexity):

$$10^{12}/10^8 = 10^4 \text{ s}$$

$$10^4 \text{ s} = 2.8 \text{ hours}$$

The heap of the next section needs about $\log_2 N$ steps per operation, where $\log_2 N$ is the power of 2 that gives N, such as 20 for a million (Section 5.2). For 10^6 inserts and removals:

$$10^6 \times 20 = 2 \times 10^7 \text{ steps}$$

$$2 \times 10^7/10^8 = 0.2 \text{ s}$$

5.2 The binary heap

A list keeps the smallest entry either hard to find or costly to maintain. A tree can keep it on top cheaply: we only demand that every entry be at most the entries just below it. Then the smallest entry is always at the top, while the rest of the tree is only partly in order, which is cheap to keep up.

A **binary heap** (G-2569) is such a tree, with two rules (DPV §4.5.2; Wikipedia, Binary heap):

- each entry has at most two entries below it, and the tree is filled level by level, left to right, with no gaps;
- every entry is at most the entries below it, so the smallest is at the top, which is what Dijkstra's algorithm needs; with the rule turned round, a heap keeps the largest entry on top instead.

Its two operations keep both rules:

- **insert:** put the new entry at the end of the bottom level, then, while it is smaller than the entry above it, swap the two (sift up);
- **remove the smallest:** take the top entry, move the last entry of the bottom level to the top, then, while it is larger than the smaller of the entries below it, swap with that one (sift down).

Figure 5 runs the heap through the pushes and removals of Dijkstra's run on the waypoint graph. In frame "insert G 3.505: sift up", G 3.505 is placed at position 3, under G 5.022; it is smaller, so the two swap, and the swap stops under B 3.000. In frame "remove the smallest: sift down", after B 3.000 comes off, the last entry G 5.022 moves to the top, is larger than G 3.505 below it, and swaps down.

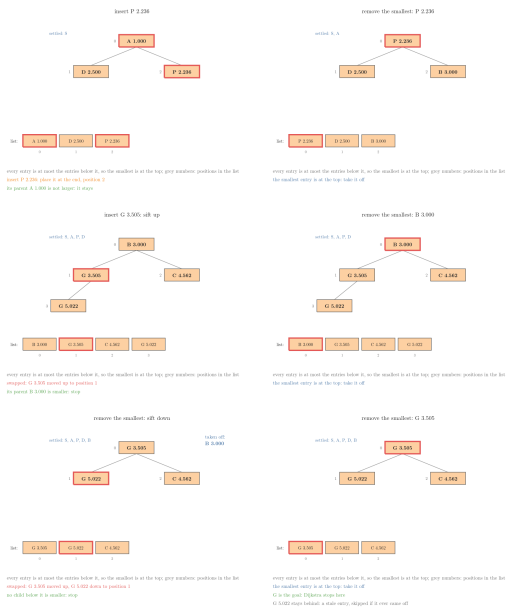


Figure 5: A binary heap of waypoints with their distances, step by step through Dijkstra's run on the waypoint graph. Top: the tree, each entry at most the entries below it; grey numbers are positions in the list. Bottom: the same entries as a list, position 0 first. Red outlines mark the entries being compared or swapped. Inserting G 3.505 places it at position 3 and sifts it up past G 5.022 to position 1, under B 3.000; removing B 3.000 moves G 5.022 to the top and sifts it down. The last frame removes G 3.505, where Dijkstra's algorithm stops, leaving the stale entry G 5.022 behind

The tree is stored in a plain list, level by level (Wikipedia, Binary heap). The symbols:

- i is the position of an entry in the list, starting at 0 for the top;
- j is the position of an entry below the top.

The two entries below position i sit at:

$$2i + 1 \quad \text{and} \quad 2i + 2$$

The entry above position j sits at the whole-number part of:

$$(j - 1)/2$$

In Figure 5, G 3.505 enters at position 3, and the entry above it is at:

$$(3 - 1)/2 = 1$$

The entry at position 1 is G 5.022, which is larger, so the two swap. From position 1, the entry above is at:

$$(1 - 1)/2 = 0$$

The entry at position 0 is B 3.000, which is smaller, so the sifting stops.

Why about $\log_2$ N steps. A sift walks one path between the top and the bottom, one level per swap, so it takes at most as many swaps as the heap has levels. The levels hold 1, 2, 4, 8 entries and so on, each twice the one above. Adding them is the sum of the levels of a search tree, with a branching factor, the number of choices per step, of 2. The symbols:

- k is the number of full levels, 3 in the small check;
- n is the number of entries they hold.

Our numbers first, then the same steps in symbols:

$$n = 1 + 2 + 4$$

$$n = 1 + 2 + \cdots + 2^{k-1}$$

$$n = 7 = 2^3 - 1$$

$$n = \frac{2^k - 1}{2 - 1} = 2^k - 1$$

Every extra level doubles the room, so N entries need only about $\log_2 N$ levels. A million entries fit in 20 levels, because:

$$2^{20} = 1{,}048{,}576$$

So insert and remove each take at most about 20 swaps for a million entries, instead of up to a million steps for a list.

5.3 Dijkstra's algorithm with a heap

When step 3 lowers a vertex's distance, as for G when D was settled, the vertex's entry in the heap should go down too. The simplest way, used with Python's `heapq` module, is not to change the old entry at all: push a new entry with the new distance, and when an entry comes off for a vertex already settled, skip it (Red Blob, implementation).

In Figure 5, G 5.022 is such a stale entry: it is still in the heap when G 3.505 comes off. Updating the entry in place also works, but needs a table from each vertex to its position in the heap (Wikipedia, Dijkstra's algorithm).

Python: Dijkstra's algorithm with Python's `heapq`, which keeps a heap with the smallest entry on top in a plain list: `heappush` inserts and sifts up, `heappop` removes the smallest and sifts down. `neighbours(v)` returns each neighbour of `v` with the cost of the edge.

```
import heapq, math
def dijkstra(start, goal, neighbours):
    dist, parent = {start: 0.0}, {start: None}
    queue, settled = [(0.0, start)], set()
    while queue:
        d, v = heapq.heappop(queue)
        if v in settled:
            continue                          # a stale entry
        settled.add(v)
        if v == goal:
            break
        for n, cost in neighbours(v):
            if d + cost < dist.get(n, math.inf):
                dist[n], parent[n] = d + cost, v
                heapq.heappush(queue, (dist[n], n))
    return dist[goal]
# S to G (notebook): 3.505 on the waypoint graph,
# 3.414 on the room grid with diagonal moves
```

The cost. Each vertex is settled once, and step 3 looks at each edge at most once from each end, so the heap sees about one insert per edge and one removal per insert. The symbols:

- V is the number of vertices, E the number of edges;
- each heap operation takes about $\log_2 V$ steps.

So Dijkstra's algorithm with a heap takes $O((V + E) \log V)$ steps, in the Big-O notation (G-2556) that keeps only the growth rate; with a plain list it takes $O(V^2)$ (DPV §4.5; Wikipedia, Dijkstra's algorithm). For a grid of 10^6 cells with diagonal moves, about 4 edges per cell:

$$(10^6 + 4 \times 10^6) \times 20 = 10^8$$

$$10^8 / 10^8 = 1 \text{ s}$$

With a plain list, V^2 is 10^{12} steps, the 2.8 hours of Section 5.1.

The heap makes each step of Dijkstra's algorithm cheap, but not the number of steps. The search still spreads in rings in every direction, with no idea where the door is: it settled 54 cells to reach the door's, including cells along the bottom wall that lead away from it.

6. Summary

Search	Frontier hands out	Finds	Needs	Work
breadth-first	the oldest vertex (queue)	the fewest moves	every edge the same cost	$O(V+E)$
Dijkstra's algorithm	the smallest distance so far (priority queue)	the shortest route	every edge cost at least 0	$O((V+E)\log V)$ with a heap
uniform-cost search	the same	the shortest route to the goal	the same	at most Dijkstra's: it stops once the goal is settled

- With diagonal moves, the robot's shortest route to the door is 3.414 m (2 diagonal and 4 straight moves), where breadth-first search returned 3.828 m, because breadth-first search counts moves and cannot tell two 6-move routes apart, while **Dijkstra's algorithm** (G-2565) counts metres.
- In an **8-connected grid** (G-2564) a diagonal move is 0.707 m by Pythagoras' rule, so the grid becomes a **weighted graph** (G-2563), and a route's cost is the sum of its edge costs.
- Dijkstra's algorithm works like a fire spreading at 1 m per second: each vertex settles at its shortest distance, so the waypoints settle in the order S 0, A 1.000, P 2.236, D 2.500, B 3.000, G 3.505, because the earliest flame to arrive came by the shortest route.
- Each settled vertex offers its neighbours the route through it, and a neighbour keeps the shorter, so G's distance dropped from 5.022 to 3.505 when D settled.
- A settled distance is final, because any other route leaves the settled vertices through a queue entry that is already at least as large and adds only lengths of at least 0; with negative costs that fails.
- **Uniform-cost search** (G-2567) stops when the goal comes off the queue, not when it first enters, because G's first entry, 5.022, was wrong.
- A **priority queue** (G-2568) hands out the smallest entry; kept as a **binary heap** (G-2569), a tree with every entry at most the entries below it, each insert or removal walks one path of about $\log_2 N$ levels for N entries, so a million entries take about 0.2 s instead of hours with a list.

The robot can now find the shortest route to the door on any graph whose edges have costs. Still missing: Dijkstra's algorithm spreads equally in every direction and settled 54 cells to reach the door; A* search adds an estimate of the distance still to go, so the search heads for the door.

7. Sources

Built from

- NPTEL, *Design and Analysis of Algorithms* (Chennai Mathematical Institute), "Single source shortest paths: Dijkstra's algorithm", YouTube, https://www.youtube.com/watch?v=OGsPQy-lM1w (NPTEL DAA, Dijkstra)
- NPTEL, *Design and Analysis of Algorithms* (Chennai Mathematical Institute), "Dijkstra's algorithm: analysis", YouTube, https://www.youtube.com/watch?v=nieg8XQQ7i0 (NPTEL DAA, Dijkstra analysis)
- NPTEL, *Design and Analysis of Algorithms* (Chennai Mathematical Institute), "Priority queues", YouTube, https://www.youtube.com/watch?v=ImSRt7LxQnY (NPTEL DAA, priority queues)
- NPTEL, *Design and Analysis of Algorithms* (Chennai Mathematical Institute), "Heaps", YouTube, https://www.youtube.com/watch?v=uQ3mBbvPToQ (NPTEL DAA, heaps)
- NPTEL, *Design and Analysis of Algorithms* (Chennai Mathematical Institute), "Heaps: Updating values, sorting", YouTube, https://www.youtube.com/watch?v=51pyj1NWEqI (NPTEL DAA, heaps: updating values)
- NPTEL, *Design and Analysis of Algorithms* (Chennai Mathematical Institute), "Introduction and motivation", YouTube, https://www.youtube.com/watch?v=JzjkWMTPkdA (NPTEL DAA, introduction)
- Patel, A. (Red Blob Games), "Introduction to the A* Algorithm", https://www.redblobgames.com/pathstar/introduction.html, and "Implementation of A*", https://www.redblobgames.com/pathfinding/a-star/implementation.html (Red Blob, introduction; Red Blob, implementation)
- UC Berkeley CS188, *Introduction to Artificial Intelligence*, online textbook, §1.3.3 "Uniform Cost Search", https://inst.eecs.berkeley.edu/~cs188/textbook/search/uninformed.html (CS188 §1.3.3)
- LaValle, S. M. (2006). *Planning Algorithms*. Cambridge University Press. §2.2.2 "Particular forward search methods": breadth first, Dijkstra's algorithm. Free online: https://lavalle.pl/planning/node44.html (LaValle §2.2.2)

Other references

- Dasgupta, S., Papadimitriou, C. H. and Vazirani, U. V. (2008). *Algorithms*. McGraw-Hill. Chapter 4: §4.4.1 the alarm-clock picture of Dijkstra's algorithm, §4.4.2 why it is correct, §4.5 priority queues and the binary heap, §4.6.1 negative edges. Archived free copy: https://web.archive.org/web/2022/https://people.eecs.berkeley.edu/~vazirani/algorithms/chap4 (DPV §4.4.1, §4.4.2, §4.5, §4.5.2, §4.6.1)
- Wikipedia, "Dijkstra's algorithm": history (conceived 1956, published 1959), proof of correctness, running time with a binary heap, stale entries instead of decrease-key and the vertex-to-position table it otherwise needs, uniform-cost search, negative weights, https://en.wikipedia.org/wiki/Dijkstra%27s_algorithm (Wikipedia, Dijkstra's algorithm)
- Wikipedia, "Priority queue": its two operations, and an unsorted or sorted array as a slow priority queue, https://en.wikipedia.org/wiki/Priority_queue (Wikipedia, Priority queue)
- Wikipedia, "Binary heap": the heap stored in a list, entries below position i at 2i + 1 and 2i + 2, https://en.wikipedia.org/wiki/Binary_heap (Wikipedia, Binary heap)
- USACO Guide, "Time Complexity": about 10^8 simple operations per second, https://usaco.guide/bronze/time-comp (USACO Guide, time complexity)

- Likhachev, M. (Carnegie Mellon University), *16-350 Planning Techniques for Robotics*, slides "Search Algorithms: Heuristics, Backward A*, Weighted A* Search": 4-connected and 8-connected grids, https://www.cs.cmu.edu/~maxim/classes/robotplanning/lectures/heuristicswei (CMU 16-350, heuristics)

8. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
8-connected grid (G-2564)	A grid made into a graph by joining each cell to all 8 cells around it: 4 straight moves of one cell side and 4 diagonal moves of 1.414 cell sides (0.707 m for 0.5 m cells), usually without cutting the corner of a blocked cell; it gives shorter, less jagged routes than a 4-connected grid.
Weighted graph (edge cost) (G-2563)	A graph in which every edge carries a number, its cost, such as a move's length in metres (0.5 m straight, 0.707 m diagonal); the cost of a path is the sum of its edge costs, so the cheapest route need not have the fewest edges.
Priority queue (G-2568)	A collection with two operations, insert an entry with a priority and remove the entry with the best priority (for Dijkstra's algorithm, the smallest distance); it is what lets a search always pick its most promising vertex next.
Dijkstra's algorithm (G-2565)	A search that always settles the vertex with the smallest distance from the start found so far, which is then final, and offers each of its neighbours the route through it, keeping the shorter; with every edge cost at least 0 it finds the shortest route to every vertex (3.414 m to the door on the room grid with diagonals).
Uniform-cost search (G-2567)	Dijkstra's algorithm stopped as soon as the goal is taken off the priority queue, the name artificial intelligence books use; it finds the shortest route to the goal without settling the vertices farther away.

Term	Meaning
Binary heap (G-2569)	A tree filled level by level in which every entry is at most the entries below it, stored in a plain list; the smallest entry sits at the top and insert and remove each walk one path of about $\log_2 N$ levels, so it makes a fast priority queue.
Breadth-first search (BFS) (G-2550)	A search whose frontier is a queue, so it expands vertices in rings of equal moves from the start; the first route it finds has the fewest moves, which is the shortest only when every move costs the same.
4-connected grid (G-2544)	A grid made into a graph by joining each cell to the up to four cells that share a side with it (left, down, up, right); the simplest graph of a grid map, with every move the same length.
Euclidean distance (G-715)	The straight-line distance between two points.
Path (graph) (G-2543)	A chain of edges in which each edge starts where the last one ended, such as S, the cell above S and the cell above that; a route the robot can follow, and what a search returns.
Parent (back pointer) (G-2576)	The vertex from which a search reached a vertex, stored with it during the search; following parents back from the goal gives the route. In A*, a cell's parent changes whenever a shorter way in lowers its cost so far.
Frontier (open list) (G-2547)	The vertices a search has reached but not yet expanded, waiting to be taken off one at a time; which one comes off next is what makes one search differ from another.
Visited list (closed list) (G-2548)	The vertices a search has already expanded; a search never expands them again, so it explores each vertex once (7 expansions on the waypoint graph instead of 40 paths).

Big-O notation (G-2556)	A way to write how fast an algorithm's work grows with the problem's size, keeping only the fastest-growing part and dropping constant factors, such as $O(V + E)$ for breadth-first search and $O(b^d)$ for listing every path; it lets us compare methods without timing them.